# Insecure by Design: A Hands-On Learning Platform for Web Security Training

Midterm Report

CSIS 4495-003 | Applied Research Project

Winter 2026

Instructor: Arasanipalai Kandhadai, Padmapriya

Check the Demonstration Video

Allwyn Mascarenhas | 300381351 | Team Leader

Thilan Udahage Don | 300398798

# Table of Contents

Introduction ..... 3
Summary of Initially Proposed Project ..... 4
Changes to the Proposal ..... 4
Adjustment to Vulnerability Scope ..... 4
Project Planning and Timeline ..... 5
Project Timeline ..... 5
Team Responsibilities ..... 5
Project Management ..... 6
Implemented Features ..... 7
1. SQL Injection (SQLi) Module - Implemented by Allwyn (Backend) & Enhanced by Thilan (UI/UX) ..... 7
2. Reflected Cross-Site Scripting (RXSS) Module - Implemented by Thilan with backend adjustments by Allwyn ..... 11
AI Use Section ..... 16
Table of AI Tools and Specific Use ..... 16
Appendix ..... 16
Work Log ..... 16
References ..... 23

# Introduction

Web application security remains a critical area of study as modern systems increasingly rely on complex, interconnected web technologies. While theoretical resources exist, many learners struggle to understand how vulnerabilities emerge in real workflows without hands-on experimentation. Practical exposure is essential for developing the ability to recognize, exploit, and mitigate security flaws; skills foundational to cybersecurity, software development, and digital forensics.

Existing vulnerable training platforms such as DVWA, WebGoat, and OWASP Juice Shop provide valuable learning opportunities, but they often rely on outdated architectures or lack structured guidance. As noted in the proposal, "many rely on outdated architectures, lack structured guidance, or do not reflect the design patterns and frameworks commonly used in contemporary enterprise applications."

Industry research reinforces this need. According to Fortinet's 2024 Cybersecurity Skills Gap Report, "58% of IT decision-makers identify a lack of hands-on cybersecurity skills and training among staff as the leading cause of security breaches."

This project addresses that gap by designing an intentionally vulnerable Spring Boot MVC application that embeds realistic vulnerabilities into functional workflows. The research explores:

- How intentional vulnerabilities can be embedded safely and educationally
- Whether guided exploration improves understanding of secure coding
- Which vulnerabilities are most effective for foundational learning

The initial hypothesis is that interacting with vulnerabilities in realistic workflows improves comprehension, engagement, and retention.

## Summary of Initially Proposed Project

The original project proposed the creation of Insecure by Design, a modern, intentionally vulnerable web application built using Spring Boot MVC. The goal was to provide a hands-on learning platform that embeds common vulnerabilities into realistic workflows, enabling learners to explore, exploit, and understand security flaws in a safe environment.

The proposal outlined:

- A design-and-implementation research methodology
- A modern technology stack (Spring Boot, H2/MySQL, Thymeleaf, HTML/CSS/JS)
- A set of planned vulnerabilities including SQL Injection, XSS, Broken Authentication, CSRF, IDOR, and File Upload flaws
- A structured timeline from Week 1 to Week 13
- Clear division of responsibilities between backend and frontend roles

## Changes to the Proposal

At this stage of the project, the core direction remains aligned with the original proposal. However, the following adjustments were made:

**Adjustment to Vulnerability Scope**

During development, we realized that the XSS and SQL Injection modules required more time than originally estimated, particularly in designing realistic workflows, implementing safe and vulnerable modes, and ensuring the educational clarity of each demonstration. To maintain quality and avoid delivering a partially implemented feature, we decided to defer the Broken Authentication module to the post-midterm phase. This adjustment allows us to deliver two fully functional, polished, and academically effective modules for the midterm, while still keeping the overall project aligned with the original proposal.

# Project Planning and Timeline

**Project Timeline**

| Week | Milestones | Deliverables |
|---|---|---|
| Week 7 — Feb 24 | Implement additional vulnerabilities (Reflected XSS); Conduct internal testing; Record midterm demo video | Midterm Report + Video Archive + Working Implementation (Due Feb 23, 11:59 PM) |
| Week 8 — Mar 3 | Implement Stored XSS; Broken Auth | Progress Report + Research/Implementation 2 (Due Mar 2, 11:59 PM) |
| Week 9 — Mar 10 | Implement CSRF vulnerability; Improve UI/UX | Progress Report + Research/Implementation 3 (Due Mar 9, 11:59 PM) |
| Week 10 — Mar 17 | Implement IDOR and File Upload vulnerability; Begin writing vulnerability walkthroughs | Progress Report + Research/Implementation 4 (Due Mar 16, 11:59 PM) |
| Week 11 — Mar 24 | Full system testing; Finalize all vulnerabilities; Begin drafting final report | Progress Report + Research/Implementation 5 (Due Mar 23, 11:59 PM) |
| Week 12 — Mar 31 | Finalize documentation; Prepare final presentation slides; Clean and organize GitHub repo | |
| Week 13 — Apr 7 | Deliver final defense presentation; Submit final report and implementation | Final Report + Implementation (Due Apr 8, 11:59 PM) |

**Team Responsibilities**

<u>Allwyn Mascarenhas (Team Lead)</u>

- Backend architecture and Spring Boot implementation
- Database design (H2/MySQL)
- Backend-driven vulnerabilities (SQL Injection, Broken Authentication, IDOR)
- Midterm and final demo coordination

Thilan Udahage Don

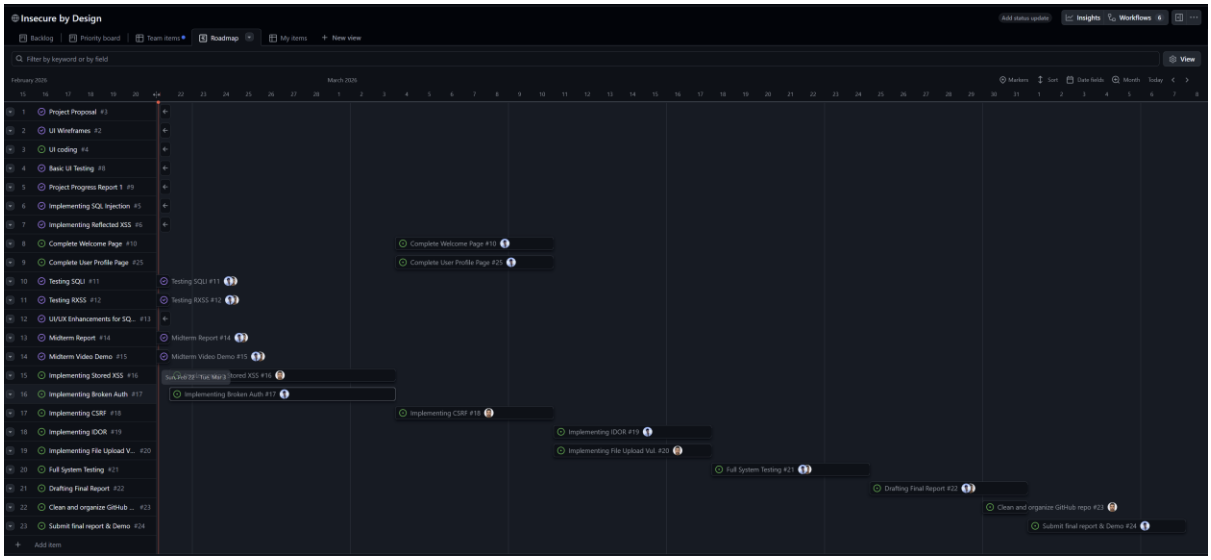
- Frontend development (Thymeleaf, HTML, CSS, JS)
- Guided learning flow and UI/UX
- Frontend-related vulnerabilities (XSS, CSRF, File Upload flaws)
- System testing and vulnerability walkthroughs

Both members collaborate on:

Vulnerability research, GitHub repo management and project board, testing, documentation (proposal, midterm report, final report) video recording, and final presentation.

**Project Management**

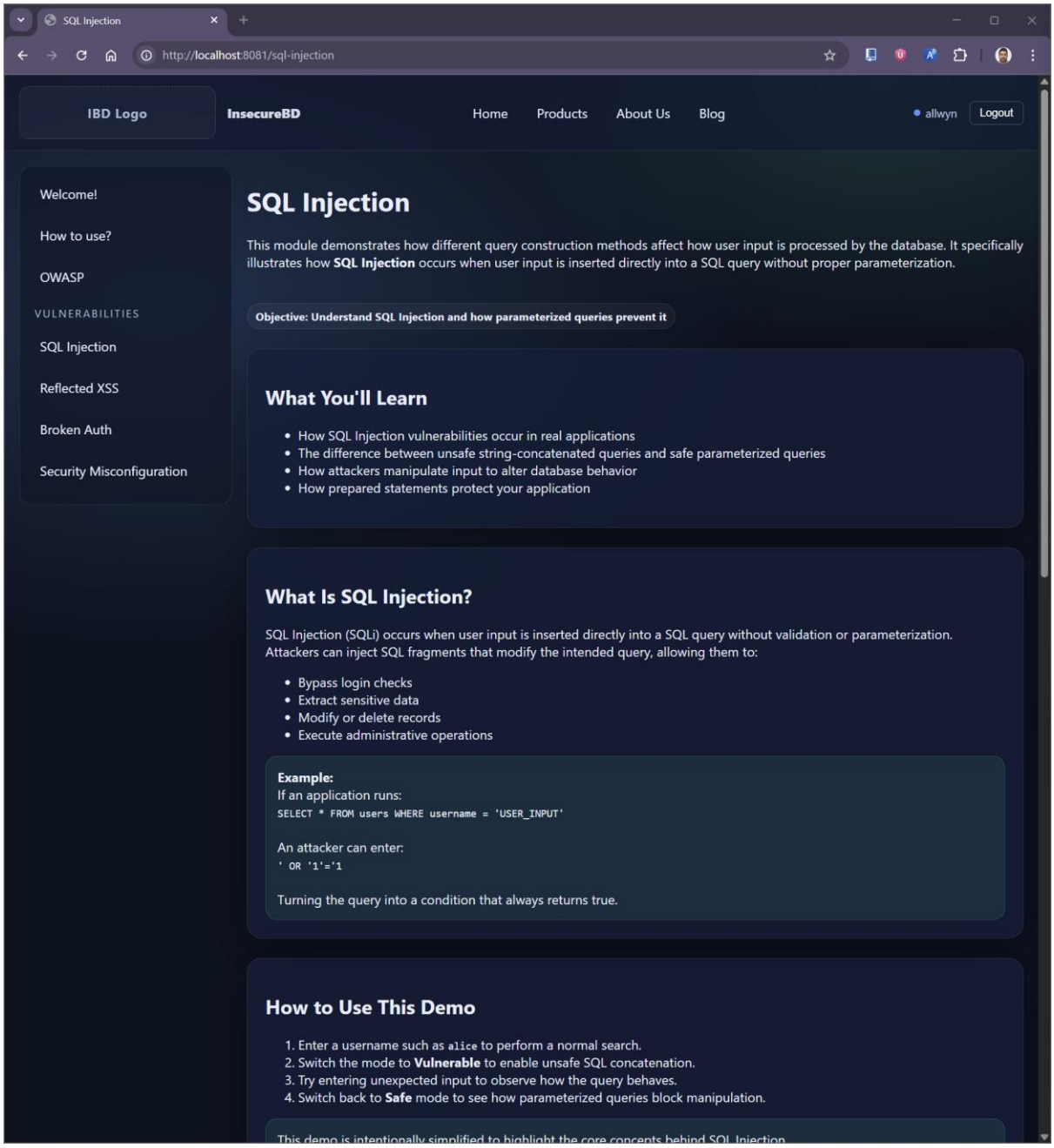
The project tasks will be tracked and assigned using GitHub projects. Tasks that need to be completed are as follows,



# Implemented Features

1. **SQL Injection (SQLi) Module - Implemented by Allwyn (Backend) & Enhanced by Thilan (UI/UX)**

The SQL Injection module demonstrates how unsafe SQL query construction can be exploited and how parameterized queries prevent manipulation. The module includes safe/vulnerable modes, a search interface, executed query display, and example payloads.



Allwyn's Contributions (Primary Developer of SQLi Module)

Backend Logic & Vulnerability Implementation

- Implemented the SQL Injection controller
- Handling of and parameters
- Safe mode using prepared statements
- Vulnerable mode using string concatenation
- Implemented the query execution logic and integrated it with the demo database
- Return error messages for malformed SQL payloads

```java
@Controller
public class SqlInjectionController {

    private final SqlInjectionService service;  3 usages

    public SqlInjectionController(SqlInjectionService service) { this.service = service; }

    @GetMapping("/sql-injection")
    public String page(
            @RequestParam(value = "q", required = false) String q,
            @RequestParam(value = "mode", defaultValue = "vuln") String mode,
            Model model
    ) {
        // If user clicked "Clear Input" → no q parameter at all
        if (q == null) {
            model.addAttribute( attributeName: "q",  attributeValue: "");
            model.addAttribute( attributeName: "mode",  attributeValue: "safe");
            model.addAttribute( attributeName: "isVulnerable",  attributeValue: false);
            model.addAttribute( attributeName: "results", List.of());
            return "pages/sql-injection";
        }

        String query = q.trim();
        boolean isVulnerable = mode.equalsIgnoreCase( anotherString: "vuln");

        // If q is empty → no results
        if (query.isBlank()) {
            model.addAttribute( attributeName: "q",  attributeValue: "");
            model.addAttribute( attributeName: "mode", mode);
            model.addAttribute( attributeName: "isVulnerable", isVulnerable);
            model.addAttribute( attributeName: "results", List.of());
            return "pages/sql-injection";
        }

        List<DemoUser> results = List.of();

        try {
            results = isVulnerable
                    ? service.vulnerableFindByUsername(query)
                    : service.safeFindByUsername(query);

        } catch (Exception e) {
            // Error message for malformed SQL
            model.addAttribute( attributeName: "error",  attributeValue: "Invalid SQL syntax or injection attempt.");
        }

        model.addAttribute( attributeName: "q", query);
        model.addAttribute( attributeName: "mode", mode);
        model.addAttribute( attributeName: "isVulnerable", isVulnerable);
        model.addAttribute( attributeName: "results", results);

        return "pages/sql-injection";
    }
```

Security Demonstration Logic:

- Implemented the safe vs. vulnerable mode switch.

• Added backend support for:

- Query preview
- Error handling
- Example SQLi payloads

Testing & Debugging

- Verified SQLi behavior across multiple payloads.
- Ensured safe mode correctly blocks injection attempts.

<u>Thilan's Contributions (UI/UX, Layout, Educational Enhancements)</u>

UI & Layout Improvements

Redesigned the SQLi page using:

- Card-based layout
- Clear spacing and typography
- Professional educational structure

Improved readability of:

- Query display
- Results table
- Error messages
- Example payloads

Custom Dropdown Component:

- Built a custom dropdown for selecting Safe/Vulnerable mode.

Added:

- Click-outside closing behavior
- Smooth interaction
- Hidden input synchronization

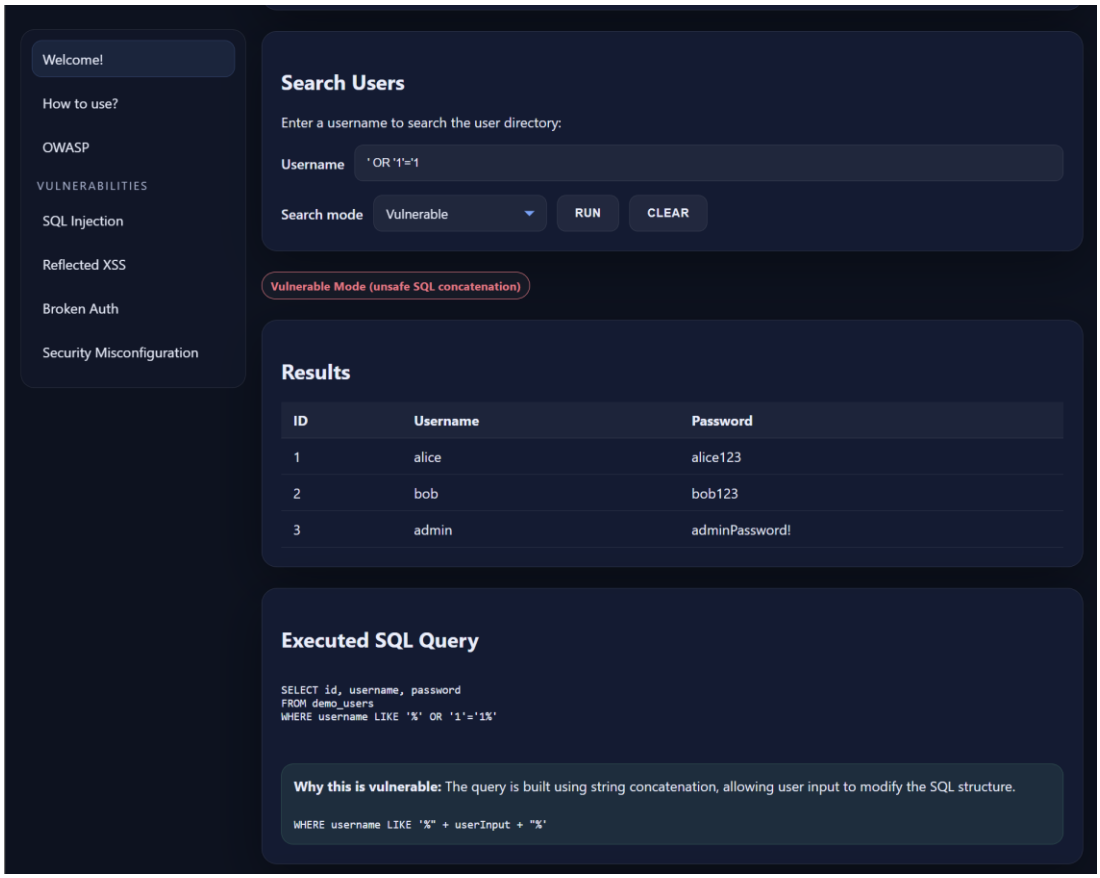
Scroll Behavior:

- Implemented a shared scroll helper script
- No scroll on initial load
- Scroll to results only when a real query is submitted
- Scroll to search form when input is empty
- Extracted this into for reuse across modules.

```
function scrollOnSearch({ queryParam : string = "q", searchSectionId, resultsSectionId }) : void { Show usages
    const params : URLSearchParams = new URLSearchParams(window.location.search);
    const queryValue : string = params.get(queryParam);

    const searchSection : HTMLElement = document.getElementById(searchSectionId);
    const resultsSection : HTMLElement = document.getElementById(resultsSectionId);

    // CASE 1: Initial load → no scroll
    if (!params.has(queryParam)) {
        return;
    }

    // CASE 2: User clicked Run with empty input → scroll to search form
    if (!queryValue || queryValue.trim() === "") {
        if (searchSection) {
            searchSection.scrollIntoView( arg: { behavior: "smooth" });
        }
        return;
    }

    // CASE 3: User submitted a real query → scroll to results
    if (resultsSection) {
        resultsSection.scrollIntoView( arg: { behavior: "smooth" });
    }
}
```

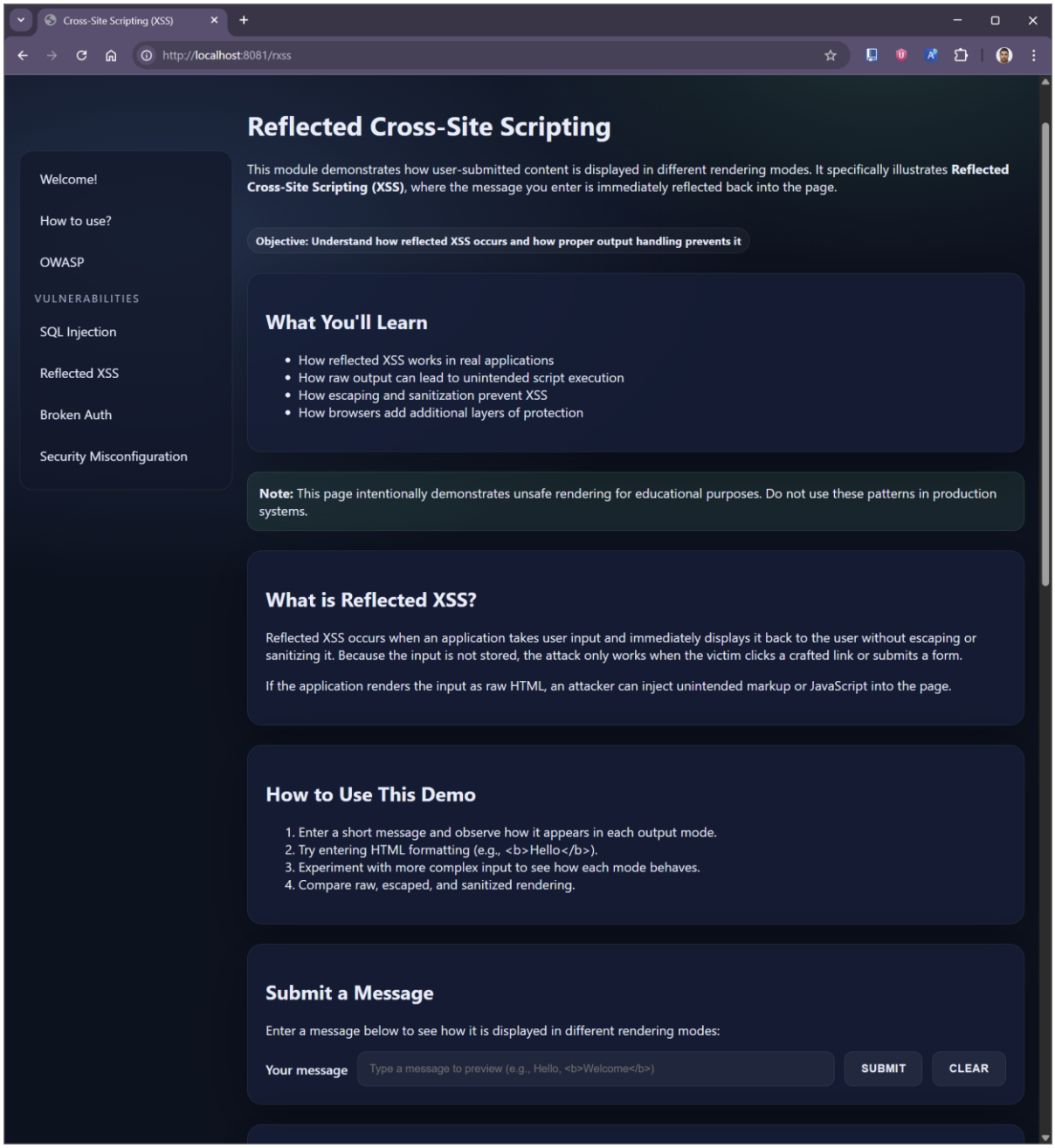
Educational Enhancements:

- Added Learning objectives
- Added Vulnerability explanations
- Added example payloads with expected behavior



2. **Reflected Cross-Site Scripting (RXSS) Module - Implemented by Thilan with backend adjustments by Allwyn**

The RXSS module demonstrates how user-supplied input is reflected back into the page using three different rendering modes: raw (unsafe), escaped (safe), and sanitized (safe). The module uses GET-based reflection, accurately simulating how real-world reflected XSS attacks occur through crafted URLs.



Thilan's Contributions (Primary Developer of RXSS Module)

1. Full Frontend & Template Implementation

Thilan designed and implemented the entire RXSS page, including:

- A clean, card-based UI consistent with the SQLi module
- Input form with Submit and Clear buttons
- Three output sections:
    - Raw Output (vulnerable) using
    - Escaped Output (safe) using
    - Sanitized Output (safe) using server-side filtering
- Educational callouts explaining:
    - Why raw output is dangerous
    - How escaping prevents XSS
    - How sanitization works
- Code snippet examples for each rendering mode
- Layout refinements, spacing, typography, and instructional clarity

This work ensured the RXSS module is visually polished, easy to understand, and academically effective.

```
    <!-- Output Comparison -->
    <div id="results-section" class="card mt-24">
        <h2>Output Comparison</h2>
        <p>This section shows how the same message is displayed using different rendering approaches.</p>

        <!-- Raw Output -->
        <div class="section-unsafe mt-24">
            <h3>1. Raw Output (vulnerable)</h3>
            <div class="card mt-16">
                <span th:utext="${input ?: ''}"></span>
            </div>

            <div class="callout mt-16">
                <b>Why this is vulnerable:</b> <code>th:utext</code> inserts the content directly into the page
                without escaping. This is what causes reflected XSS.
            </div>

            <pre class="mt-16"><code>&lt;span th:utext="${input}"&gt;&lt;/span&gt;</code></pre>
        </div>

        <!-- Escaped Output -->
        <div class="section-safe mt-24">
            <h3>2. Escaped Output (safe)</h3>
            <div class="card mt-16">
                <span th:text="${input ?: ''}"></span>
            </div>

            <div class="callout callout-info mt-16">
                <b>Why this is safe:</b> <code>th:text</code> escapes HTML characters before rendering.
                This prevents XSS by treating the input as plain text.
            </div>

            <pre class="mt-16"><code>&lt;span th:text="${input}"&gt;&lt;/span&gt;</code></pre>
        </div>

        <!-- Sanitized Output -->
        <div class="section-safe mt-24">
            <h3>3. Sanitized Output (safe)</h3>
            <div class="card mt-16">
                <span th:text="${safeInput ?: ''}"></span>
            </div>

            <div class="callout callout-info mt-16">
                <b>Why this is safe:</b> The server removes or replaces potentially unsafe characters.
            </div>

            <pre class="mt-16"><code>String sanitized = input
    .replace("&lt;", "&amp;lt;")
    .replace("&gt;", "&amp;gt;");
model.addAttribute("safeInput", sanitized);</code></pre>
        </div>
    </div>
```

2. GET-Based Reflection Workflow

Thilan implemented the full GET-based reflection workflow:

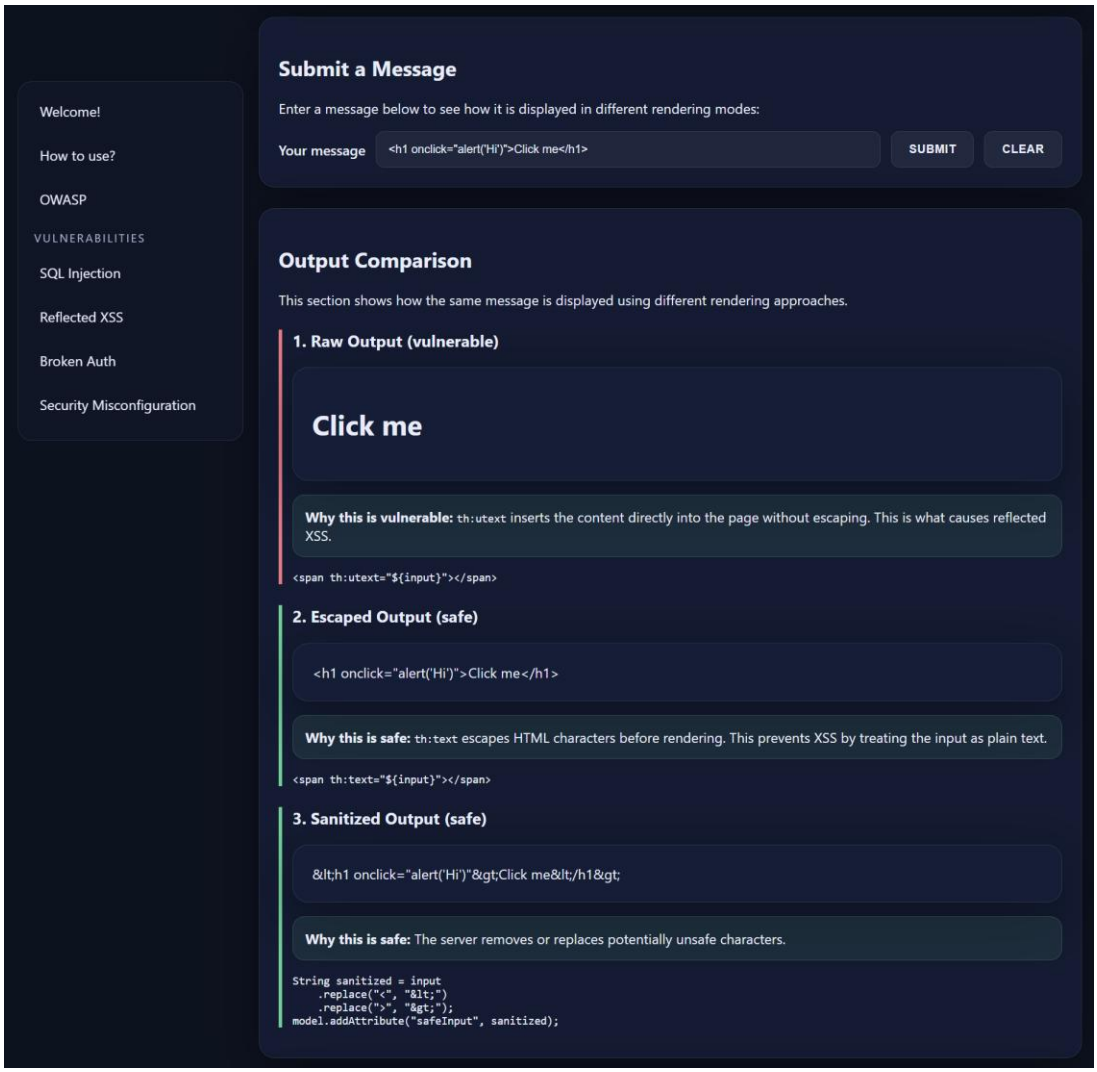
- Input is passed via URL query parameters
- The page reflects the input into all three output modes
- This accurately simulates real reflected XSS attacks
- Ensures the module behaves like real-world vulnerable applications

3. Scroll Behavior Integration

Thilan integrated the shared scroll helper script into the RXSS module:

- Scrolls to output only when input exists
- Scrolls to the form when input is empty
- Does not scroll on initial load
- Ensures consistent UX across modules

This required adjusting the RXSS template structure and IDs to match the helper's configuration.



4. UI/UX Consistency Across Modules

Thilan ensured the RXSS module matched the SQLi module's:

- Card layout
- Typography
- Badge components
- Section spacing
- Educational structure

This created a unified learning experience across the platform.

Allwyn's Contributions to RXSS

Although the RXSS module was primarily implemented by Thilan, Allwyn contributed important backend improvements:

1. CSP Header Adjustment

Originally, Thilan added:

*response.setHeader("Content-Security-Policy", "default-src 'self'")*

This blocked scripts entirely, preventing the raw XSS payload from executing, which defeats the purpose of demonstrating reflected XSS.

Allwyn corrected this by changing it to:

*response.setHeader("Content-Security-Policy-Report-Only", "default-src 'self'")*

This allows:

- The browser to report violations
- But still execute the payload for demonstration purposes

This was essential for making the RXSS module function as intended.

2. Created a Separate RXSS Controller

To improve project structure and maintainability, Allwyn:

- Extracted RXSS logic into a dedicated
- Cleaned up routing and separated it from other modules
- Ensured the controller correctly handled GET-based reflection
- Updated model attributes for raw, escaped, and sanitized output

This made the module cleaner, more modular, and easier to maintain.

```java
@Controller
public class RxssController {

    // Report-Only so XSS payload still executes
    @ModelAttribute
    public void addSecurityHeaders(HttpServletResponse response) {
        response.setHeader( s: "Content-Security-Policy-Report-Only",  s1: "default-src 'self'");
        // BLOCK XSS (hard mode)
        // response.setHeader("Content-Security-Policy", "default-src 'self'");
    }

    @GetMapping("/rxss")
    public String xssPage(@RequestParam(required = false) String input, Model model) {

        // Raw input for th:utext and th:text
        model.addAttribute( attributeName: "input", input);

        // Sanitized version
        if (input != null && !input.isBlank()) {
            String sanitized = input
                    .replace( target: "<",  replacement: "&lt;")
                    .replace( target: ">",  replacement: "&gt;");
            model.addAttribute( attributeName: "safeInput", sanitized);
        }

        return "pages/rxss";
    }
}
```

# AI Use Section

**Table of AI Tools and Specific Use**

| AI Tool | Version | Feature Used | Value Add |
|---|---|---|---|
| Chatgpt | 5.1 free | Grammar and writing help | Used only as a writing aid to flesh out original ideas |
| Claude | 4.5 free | Prototyping and coding | Used to generate quick mocks before making final commitment to the design. |

**Appendix**

# Work Log

Allwyn Mascarenhas

| Date | Number of Hours | Description of Work |
|---|---|---|
| 15th Jan | 2 | Conducted initial research on OWASP Top 10 vulnerabilities and reviewed documentation for DVWA, WebGoat, and Juice Shop. Compared their learning approaches and identified opportunities for a simpler, framework-based platform. |
| 20th Jan | 1 | Participated in instructor-led project consultation. Discussed project direction, technology stack, and how to embed vulnerabilities safely within a Spring Boot MVC application. |
| 21st Jan | 1.5 | Co-drafted the pre-proposal for instructor approval. Summarized research findings and outlined the initial problem definition, research questions, and project scope. |
| 22nd Jan | 1 | Researched Spring Boot security misconfigurations and common vulnerability patterns (e.g., insecure authentication flows, improper input validation). Documented potential implementation strategies. |
| 23rd Jan | 1 | Attended scheduled Friday team meeting. Finalized task distribution and reviewed proposal template. |
| 23rd Jan | 1.5 | Created and organized the GitHub Project Kanban board. Added tasks for proposal writing, research, |

| Date | Number of Hours | Description of Work |
|---|---|---|
| | | vulnerability implementation, and documentation. Assigned responsibilities and set up workflow columns. |
| 24th Jan | 2 | Collaborated with Thilan on drafting the full project proposal. Wrote sections on scope, methodology, and technology stack. Ensured the proposal met the rubric requirements for introduction and research design. |
| 25th Jan | 2 | Continued refining the proposal independently. Reviewed the draft for clarity, strengthened the methodology section, and added additional justification for the selected technology stack. |
| 26th Jan | 1 | Reviewed and refined the proposal draft. Added details to the methodology and expected results sections. Submitted the document to Blackboard and GitHub. |
| 27th Jan | 1 | Participated in in-class proposal check-in. Instructor requested adding industry data to justify the project. Reviewed Fortinet's 2024 Skills Gap Report and incorporated the statistic that 58% of IT decision-makers cite lack of hands-on skills as a top cause of breaches. |
| 31st Jan | 1 | Attended scheduled Friday team meeting. Discussed Week 5 implementation tasks, including setting up the Spring Boot skeleton and configuring the H2 database. |
| 1st Feb | 1.5 | Attended scheduled Saturday team meeting. Reviewed architecture outline and refined backend workflow diagrams. Finalized vulnerability placement strategy for SQL Injection, Broken Authentication, and IDOR. |
| 3rd Feb | 1 | Began setting up backend project structure. Created initial controller and service templates to prepare for SQL Injection implementation. |
| 4th Feb | 1.5 | Completed Spring Boot skeleton setup. Verified project builds successfully, configured H2 database, and tested basic backend routing. |

| Date | Number of Hours | Description of Work |
|---|---|---|
| 5th Feb | 1 | Conducted deeper research on SQL Injection exploitation in Java applications. Reviewed insecure query patterns and prepared notes for implementing the first vulnerability. |
| 6th Feb | 1 | Drafted backend-focused content for Progress Report 1, summarizing architecture planning, backend setup, and SQL Injection research. |
| 7th Feb | 1 | Updated backend folder structure and added initial placeholder files for SQL Injection implementation. Reviewed vulnerability placement documentation added by Thilan. |
| 9th Feb | 2 | Implemented the vulnerable SQL query using string concatenation. Verified that SQLi payloads produced unsafe behavior and returned unintended results. |
| 10th Feb | 1.5 | Implemented the safe SQL query using prepared statements. Ensured parameterized queries blocked injection attempts and added backend logic to toggle between safe and vulnerable modes. |
| 13th Feb | 1 | Weekly team meeting. Reviewed SQLi progress, discussed RXSS routing improvements, and planned controller refactoring for better modularity. |
| 14th Feb | 1 | Continued team meeting. Finalized SQLi backend logic and confirmed the structure for GET-based reflection in the upcoming RXSS controller. |
| 15th Feb | 1 | Added backend support for executed SQL query display and improved error handling for malformed SQL payloads. Ensured the application returned meaningful error messages without breaking. |
| 17th Feb | 1 | Created a dedicated RxssController to separate XSS logic from other modules. Cleaned up routing and ensured proper GET-based reflection behavior. |
| 18th Feb | 1 | Adjusted the CSP header from Content-Security-Policy to Content-Security-Policy-Report-Only to allow XSS payload execution for demonstration while still logging violations. |
| 20th Feb | 1 | Performed integration testing for SQLi and RXSS modules. Verified backend compatibility with UI |

| Date | Number of Hours | Description of Work |
|---|---|---|
| | | components, dropdown behavior, and scroll helper functionality. |
| 21st Feb | 1.5 | Wrote backend sections of SQLi implementation details and RXSS controller explanation. Participated in recording the midterm demonstration video. Uploaded updated code, the completed midterm report, and the demonstration video to the GitHub repository. |

Thilan Udahage Don

| Date | Number of Hours | Description of Work |
|---|---|---|
| 20th Jan | 1 | Participated in the instructor-led project consultation. Discussed feasibility of building an intentionally vulnerable Spring Boot application and clarified expectations for the proposal. |
| 20th Jan | 1 | Conducted research on existing vulnerable web applications (DVWA, WebGoat, Juice Shop). Identified gaps such as outdated frameworks and lack of structured guidance. Documented findings for proposal use. |
| 21st Jan | 1.5 | Co-drafted the pre-proposal for instructor approval. Summarized research findings and outlined the initial problem definition, research questions, and project scope. |
| 22nd Jan | 1 | Reviewed OWASP Top 10 documentation to determine which vulnerabilities would be most suitable for embedding into a Spring Boot MVC application. Took notes on implementation patterns and potential risks. |
| 23rd Jan | 1 | Attended scheduled team meeting. Discussed proposal structure, division of responsibilities, and GitHub setup tasks. |
| 23rd Jan | 1.5 | Created the GitHub repository structure following course requirements. Added folders (ReportsAndDocuments, Implementation, Misc), |

| Date | Number of Hours | Description of Work |
|---|---|---|
| | | created worklog.md and deliverables.md, updated README. |
| 24th Jan | 2 | Attended scheduled Saturday team meeting. Collaborated with Allwyn on drafting the proposal. Focused on introduction, research problem framing, and practical contributions. |
| 25th Jan | 2 | Reviewed each section for coherence, corrected structural inconsistencies, and ensured that the introduction, methodology, and expected results flowed logically. Added missing justification points and improved the clarity of research questions. |
| 26th Jan | 1 | Conducted a full review of the proposal draft to ensure consistency between the problem statement, methodology, and timeline. Verified that all sections met rubric requirements. Checked formatting, corrected grammar. Prepared the document for submission. |
| 27th Jan | 1 | Participated in in-class proposal check-in. Instructor requested that we justify the project by referencing data that highlights the need for practical security training. Reviewed industry reports, including a Fortinet article noting that many security professionals lack hands-on experience with real vulnerabilities. Added this evidence to the proposal to strengthen the problem motivation. |
| 31st Jan | 1 | Attended scheduled Friday team meeting. Discussed Week 5 tasks, including setting up the Spring Boot skeleton, configuring H2 database, and planning the first vulnerability implementation. |
| 1st Feb | 1.5 | Attended scheduled Saturday team meeting. Reviewed architecture outline and refined user flows. Finalized vulnerability placement strategy for SQLi, XSS, and Broken Auth. |
| 2nd Feb | 1 | Drafted initial documentation structure for implementation and vulnerability planning to be added to GitHub later in the week. |

| Date | Number of Hours | Description of Work |
|---|---|---|
| 4th Feb | 1.5 | Set up local Spring Boot development environment. Verified dependencies, created initial project structure, and tested basic application startup. |
| 5th Feb | 1 | Conducted research on SQL Injection patterns in Spring Boot applications. Reviewed insecure repository usage examples and documented potential implementation approaches. |
| 6th Feb | 1 | Drafted initial content for Progress Report 1, summarizing research, architecture planning, and early setup work completed so far. |
| 7th Feb | 1 | Added implementation_plan.md, vulnerability_placement.md to the GitHub repository. Documented planned workflows and frontend responsibilities. |
| 8th Feb | 1.5 | Reviewed the Spring Boot project structure and verified local environment functionality. Updated work logs and prepared remaining content for Progress Report 1. |
| 9th Feb | 1 | Finalized and formatted Progress Report 1. Checked in all documentation files and uploaded the completed report to the GitHub repository under ReportsAndDocuments. |
| 11th Feb | 1.5 | Began implementing the Reflected XSS (RXSS) module. Set up the initial page structure, input form, and output sections for raw, escaped, and sanitized rendering. |
| 12th Feb | 2.5 | Completed the RXSS UI. Added educational callouts, code snippet examples, and ensured the layout matched the SQLi module. Verified GET-based reflection behavior. Reviewed Allwyn's SQL Injection backend implementation and planned UI structure for the SQLi module to ensure consistency with future modules. |
| 13th Feb | 1 | Weekly team meeting. Reviewed RXSS progress, discussed routing cleanup, and planned remaining UI refinements for both modules. |

| Date | Number of Hours | Description of Work |
|---|---|---|
| 14th Feb | 1 | Continued team meeting. Finalized RXSS page structure and confirmed alignment with SQLi's educational layout. |
| 15th Feb | 1 | Added sanitization logic for RXSS and refined the raw/escaped/sanitized output sections. Improved clarity of code snippet explanations. |
| 17th Feb | 1 | Implemented the custom dropdown component for SQLi mode selection (Safe/Vulnerable). Added click-outside closing behavior and hidden input synchronization. |
| 18th Feb | 1.5 | Integrated the dropdown into the SQLi UI and refined spacing, typography, and card layout for both modules. Updated routing from /xss to /rxss. |
| 20th Feb | 1 | Wrote midterm report sections covering RXSS implementation, UI/UX design decisions, scroll helper functionality, and educational rationale. Prepared screenshots and code snippets for inclusion. |
| 21st Feb | 3 | Finalized the midterm report, added missing sections, formatted tables, and committed the completed code to the GitHub repository. Participated in recording the midterm demonstration video. |

# References

Fortinet. (2024). 2024 Cybersecurity Skills Gap Report. Retrieved January 2026, from https://www.fortinet.com/content/dam/fortinet/assets/reports/2024-cybersecurity-skills-gap-report.pdf

Mudge, R. (digininja). (n.d.). Damn Vulnerable Web Application (DVWA) [GitHub repository]. Retrieved January 2026, from https://github.com/digininja/DVWA

OWASP Foundation. (n.d.). OWASP Juice Shop. Retrieved January 2026, from https://owasp.org/www-project-juice-shop/

OWASP Foundation. (n.d.). OWASP WebGoat. Retrieved January 2026, from https://owasp.org/www-project-webgoat/